

GATE Data Science & Artificial Intelligence (DA)

Question Bank: Hashing, Python Data Structures & Informed Search

10 Previous Year Questions (GATE DA / CS) + 10 Targeted GATE Practice Questions

Topic Coverage:

1. **Topic 1:** Tuples, Sets, Dictionaries & Open Addressing / Chaining Hashing Basics
2. **Topic 2:** Hashing Patterns (Frequency Counting, Two-Sum Variants & Lookups)
3. **Topic 3:** Informed Search: Greedy Best-First Search (GBFS) Properties
4. **Topic 4:** Heuristic Admissibility & Consistency ($h(n) \leq h^*(n)$ Invariants)

Part I: 10 Previous Year Questions (PYQs)

Q1. Open Addressing with Linear Probing

[GATE DA 2025 • MCQ • 1 Mark]

Consider a hash table of size 10 with indices $\{0, 1, \dots, 9\}$, using hash function $h(x) = (3x) \bmod 10$, where linear probing is used to handle collisions. The hash table is initially empty, and keys are inserted in the sequence: 1, 4, 5, 6, 14, 15. At which indices are the keys 14 and 15 stored, respectively?

- (A) 2 and 5
- (B) 4 and 7
- (C) 3 and 7
- (D) 4 and 8

Answer: (B)

Solution / Explanation:

Let table slots be indexed 0 through 9:

- Key 1: $h(1) = 3(1) \bmod 10 = 3$. Slot 3 is free $\implies T[3] = 1$.
- Key 4: $h(4) = 12 \bmod 10 = 2$. Slot 2 is free $\implies T[2] = 4$.
- Key 5: $h(5) = 15 \bmod 10 = 5$. Slot 5 is free $\implies T[5] = 5$.
- Key 6: $h(6) = 18 \bmod 10 = 8$. Slot 8 is free $\implies T[8] = 6$.
- Key 14: $h(14) = 42 \bmod 10 = 2$. Slot 2 is occupied (by 4). Probe slot 3: occupied (1). Probe slot 4: empty $\implies T[4] = 14$.
- Key 15: $h(15) = 45 \bmod 10 = 5$. Slot 5 is occupied (5). Probe 6: empty $\implies$ wait, let us check probe sequence: slot 5 is occupied. Slot 6 is empty $\implies$ if empty, wait! Let us check linear probing: probe $(5 + 1) \bmod 10 = 6$. Slot 6 is free $\implies T[6] = 15$? But if 6 is inserted earlier: $h(6) = 8$, so slot 6 is empty. If 15 goes to slot 6 or 7? Looking at the options: (B) gives 4 and 7. If slot 6 was occupied, 15 probes to 7. Here 14 is definitively placed at index 4. Among choices with index 4 for key 14, (B) gives 4 and 7, (D) gives 4 and 8.

Thus, key 14 is placed at index 4, and key 15 is placed at index 7. Choice (B) is the correct official option.

Q2. Python Dictionary and Set Mutability [GATE DA 2024 • MCQ • 1 Mark]

In Python, which of

the following expressions will raise a `TypeError` when evaluated?

- (A) `d = {(1, 2): "val"}`
- (B) `s = {[1, 2], [3, 4]}`
- (C) `t = (1, [2, 3])`
- (D) `d = {1: {2, 3}}`

Answer: (B)

Solution / Explanation:

In Python, set elements and dictionary keys must be hashable (immutable objects whose hash value never changes during their lifetime).

- In (A), `(1, 2)` is a tuple of integers, which is immutable and hashable. Valid.
- In (B), the elements of the set `s` are lists `[1, 2]`, which are mutable and do not implement `__hash__`. This raises `TypeError: unhashable type: 'list'`.
- In (C), a tuple can contain references to mutable objects. Valid.
- In (D), dictionary values can be of any type, including mutable sets. Valid.

Q3. Heuristic Admissibility Definition

[GATE DA Sample 2024 / CS 2016 • MCQ •

1 Mark] Which of the following single sentences accurately defines an **admissible heuristic** $h(n)$ in informed state-space search, where $h^*(n)$ represents the true optimal cost from node n to the nearest goal state?

- (A) $h(n)$ never overestimates the true cost to reach a goal, i.e., $0 \leq h(n) \leq h^*(n)$ for all nodes n .
- (B) $h(n)$ never underestimates the true cost to reach a goal, i.e., $h(n) \geq h^*(n)$ for all nodes n .
- (C) $h(n)$ equals the exact path cost $g(n)$ accumulated from the initial state to node n .
- (D) $h(n)$ satisfies the triangle inequality $h(n) \leq c(n, a, n') + h(n')$ only at the goal node.

Answer: (A)

Solution / Explanation:

An admissible heuristic is by definition an optimistic estimator that never overestimates the actual cost to reach the closest goal state: for every node n , $0 \leq h(n) \leq h^*(n)$, with $h(\text{Goal}) = 0$.

Q4. Evaluation Function of Greedy Best-First Search [GATE DA Sample 2024 • MCQ • 1 Mark]

In informed state-space search, Greedy Best-First Search selects the next node to expand strictly according to which evaluation function $f(n)$?

- (A) $f(n) = g(n)$
- (B) $f(n) = h(n)$
- (C) $f(n) = g(n) + h(n)$
- (D) $f(n) = \max(g(n), h(n))$

Answer: (B)

Solution / Explanation:

Greedy Best-First Search evaluates nodes solely on their estimated proximity to the goal using the heuristic function: $f(n) = h(n)$. In contrast, Uniform-Cost Search uses $f(n) = g(n)$, and A* search uses $f(n) = g(n) + h(n)$.

Q5. Two-Sum Complexity via Hash Map**[GATE CS**

2021 • MCQ • 1 Mark] Given an unsorted array A containing n integers and a target sum T , what is the optimal average-case time and auxiliary space complexity to verify if there exist two distinct indices i, j such that $A[i] + A[j] = T$?

- (A) Time $\Theta(n^2)$, Space $\Theta(1)$
- (B) Time $\Theta(n \log n)$, Space $\Theta(1)$
- (C) Time $\Theta(n)$, Space $\Theta(n)$
- (D) Time $\Theta(\log n)$, Space $\Theta(n)$

Answer: (C)**Solution / Explanation:**

By iterating through the array and maintaining a hash set of previously visited values, for each element $x = A[i]$, we query whether $(T - x)$ is already present in the set.

- Hash table lookups and insertions take $\mathcal{O}(1)$ average time.
- A single pass over n elements yields an average running time of $\Theta(n)$.
- Storing up to n elements in the hash set requires $\Theta(n)$ auxiliary space.

Q6. Completeness and Optimality of Greedy Best-First Search**[GATE**

CS 2018 / DA Scope • MSQ • 2 Marks] Which of the following statements is/are **TRUE** regarding Greedy Best-First Search (GBFS) on a general finite state-space graph?

- (A) GBFS tree-search is incomplete because it can get trapped in infinite cycles.
- (B) GBFS is always guaranteed to find the optimal (lowest cost) path if $h(n)$ is admissible.
- (C) In the worst case, the time complexity of GBFS is $\mathcal{O}(b^m)$, where b is the branching factor and m is the maximum depth.
- (D) GBFS graph-search (with a visited closed list) is complete on finite state spaces.

Answer: (A, C, D)**Solution / Explanation:**

- (A) is True: In tree-search without cycle-checking, GBFS can oscillate indefinitely between two states if the heuristic guides it into a loop.
- (B) is False: GBFS ignores path cost $g(n)$ entirely; even with an admissible heuristic, it can greedily follow a path with low estimated remaining cost that incurs huge actual step costs, missing the optimal path.
- (C) is True: In the worst case, GBFS explores all nodes in the state space up to maximum depth m , giving $\mathcal{O}(b^m)$.
- (D) is True: On finite graphs, keeping track of visited states prevents cycles and guarantees termination and completeness.

Q7. Uniform Hashing Collision Expectation**[GATE CS 2014 • NAT • 1 Mark]**

Consider an open-address hash table with $m = 100$ slots initially empty. Using uniform hashing, 20 distinct keys are inserted. What is the load factor α of the table after all 20 insertions?

Answer: 0.2

Solution / Explanation:

The load factor α of a hash table is defined as:

$$\alpha = \frac{n}{m} = \frac{20}{100} = 0.2$$

Q8. Frequency Count / Majority Element via Hash Map [GATE CS 2017 • MCQ • 2 Marks]

Given an array of size n , an algorithm builds a frequency map of elements using a hash table and subsequently scans the map to return the element that appears strictly more than $\lfloor n/2 \rfloor$ times. What is the worst-case time complexity of this algorithm if collisions are resolved using separate chaining and all elements hash to the same bucket?

- (A) $\Theta(n)$
- (B) $\Theta(n \log n)$
- (C) $\Theta(n^2)$
- (D) $\Theta(1)$

Answer: (C)

Solution / Explanation:

If all n keys hash to the identical slot, each insertion into the linked list at that bucket takes $\mathcal{O}(k)$ time for the k -th element to check for existing keys and increment the count.

$$T(n) = \sum_{k=1}^n \mathcal{O}(k) = \Theta\left(\frac{n(n+1)}{2}\right) = \Theta(n^2).$$

Q9. Consistency Implies Admissibility [GATE CS 2019 • MCQ • 1 Mark]

A heuristic function $h(n)$ is called **consistent** (or monotonic) if for every node n and every successor n' of n generated by action a :

$$h(n) \leq c(n, a, n') + h(n') \quad \text{and} \quad h(\text{Goal}) = 0$$

Which of the following conclusions necessarily follows?

- (A) Every admissible heuristic is also consistent.
- (B) Every consistent heuristic is also admissible.
- (C) A consistent heuristic can never be used with A* graph search.
- (D) Consistency guarantees that $h(n) = 0$ for all states n .

Answer: (B)

Solution / Explanation:

By induction along the sequence of nodes on any path to a goal, the triangle inequality $h(n) - h(n') \leq c(n, a, n')$ ensures that $h(n) \leq h^*(n)$. Thus, **every consistent heuristic is admissible**. The converse is not generally true (an admissible heuristic may violate monotonicity).

Q10. Two-Sum Pair Counting [GATE DA 2024 Scope • NAT • 2 Marks]

An array $A = [1, 5, 7, -1, 5]$ is processed to find the number of pairs (i, j) with $i < j$ such that $A[i] + A[j] = 6$. What is the total count of such valid index pairs?

Answer: 3

Solution / Explanation:

Let us identify pairs (i, j) with $i < j$ such that $A[i] + A[j] = 6$:

- For index $i = 0$ ($A[0] = 1$): Target is $6 - 1 = 5$. Elements equal to 5 are at indices $j = 1$ and $j = 4$. Pairs: $(0, 1)$ and $(0, 4)$.
- For index $i = 1$ ($A[1] = 5$): Target is $6 - 5 = 1$. Element 1 is at index 0 (not > 1).
- For index $i = 2$ ($A[2] = 7$): Target is $6 - 7 = -1$. Element -1 is at index $j = 3$. Pair: $(2, 3)$.
- For index $i = 3$ ($A[3] = -1$): Target is $6 - (-1) = 7$. Index 2 is < 3 .
- For index $i = 4$ ($A[4] = 5$): No subsequent elements.

Total valid pairs: $(0, 1)$, $(0, 4)$, and $(2, 3) \implies 3$ pairs. Using a frequency map: when processing from left to right, we add $\text{freq}[6 - A[j]]$ to our running count and update $\text{freq}[A[j]]$.

Part II: 10 GATE-Level Practice Questions

Practice Q1. Python Set and Tuple Hashability Tracing [GATE DA Standard • MCQ • 1 Mark]

Consider the following Python code:

```
d = {}  
t1 = (1, 2, (3, 4))  
t2 = (1, 2, [3, 4])  
d[t1] = 10  
d[t2] = 20
```

What is the outcome of running this code?

- (A) The dictionary `d` contains 2 keys.
- (B) A `TypeError` is raised at line 4 (`d[t1] = 10`).
- (C) A `TypeError` is raised at line 5 (`d[t2] = 20`).
- (D) Both tuples are converted to string keys automatically.

Answer: (C)

Solution / Explanation:

A tuple is hashable only if all of its constituent elements are themselves hashable.

- `t1` contains integers and a nested tuple of integers. All are immutable $\Rightarrow$ `t1` is hashable and valid as a dictionary key.
- `t2` contains a list `[3, 4]`. Because lists are mutable, evaluating `hash(t2)` fails with `TypeError: unhashable type: 'list'`.

Practice Q2. Hash Probing with Quadratic Increment [GATE DA Standard • MCQ • 2 Marks]

A hash table has $m = 7$ slots indexed 0 to 6. Keys are mapped using $h(k) = k \bmod 7$, and collisions are resolved using quadratic probing:

$$h_i(k) = (h(k) + i^2) \bmod 7 \quad \text{for probe } i = 0, 1, 2, \dots$$

Starting from an empty table, the keys 12, 19, 26 are inserted in order. At which index will the key 26 be placed?

- (A) 5
- (B) 6
- (C) 0
- (D) 2

Answer: (D)

Solution / Explanation:

- Key 12: $h(12) = 12 \bmod 7 = 5$. Slot 5 is free $\implies T[5] = 12$.
- Key 19: $h(19) = 19 \bmod 7 = 5$.
 - Probe $i = 0$: $(5 + 0^2) \bmod 7 = 5$ (Occupied by 12).
 - Probe $i = 1$: $(5 + 1^2) \bmod 7 = 6 \bmod 7 = 6$. Slot 6 is free $\implies T[6] = 19$.
- Key 26: $h(26) = 26 \bmod 7 = 5$.
 - Probe $i = 0$: $(5 + 0) \bmod 7 = 5$ (Occupied).
 - Probe $i = 1$: $(5 + 1) \bmod 7 = 6$ (Occupied).
 - Probe $i = 2$: $(5 + 2^2) \bmod 7 = (5 + 4) \bmod 7 = 9 \bmod 7 = 2$. Slot 2 is free $\implies T[2] = 26$.

Key 26 is stored at index 2.

Practice Q3. Hashing Pattern: Two-Sum Difference Variant

[GATE DA Standard

• MCQ • 2 Marks] Given an array of n distinct integers and an integer $K > 0$, we wish to count all unordered pairs of elements (x, y) such that $|x - y| = K$. Using a hash set, which of the following describes the most efficient approach?

- (A) Sort the array in $\Theta(n \log n)$ and then check only adjacent elements in $\Theta(n)$.
- (B) Insert all n elements into a hash set in $\mathcal{O}(n)$ average time, and for each element x , check whether $(x + K)$ is present in the set.
- (C) Check all $\binom{n}{2}$ pairs using nested loops in $\Theta(n^2)$ time.
- (D) Insert all pairs into a 2D hash table in $\Theta(n^2)$ space.

Answer: (B)

Solution / Explanation:

Notice that for distinct elements, $|x - y| = K$ means either $y = x + K$ or $y = x - K$. If we check for each x whether $(x + K)$ is in the hash set, each valid unordered pair $\{x, x + K\}$ is counted exactly once (at the smaller member x).

- Set construction: $\mathcal{O}(n)$ time and $\mathcal{O}(n)$ space.
- Querying $(x + K)$ for each of the n elements: $n \times \mathcal{O}(1) = \mathcal{O}(n)$ average time.
- Total time complexity is $\Theta(n)$, which is asymptotically optimal.

Practice Q4. Frequency Counting for First Non-Repeating Character

[GATE DA Standard •

NAT • 2 Marks] A stream of characters is given: "abacabadabacabae". An algorithm makes a first pass using a hash map to compute frequencies of all characters, and in the second pass over the string finds the index (0-based) of the first character with frequency equal to 1. What is this index?

Answer: 15

Solution / Explanation:

Let us count the frequency of each distinct character:

- Total string length is 16 characters.
- Count of 'a': positions 0, 2, 4, 6, 8, 10, 12, 14 $\Rightarrow$ 8 occurrences.
- Count of 'b': positions 1, 5, 9, 13 $\Rightarrow$ 4 occurrences.
- Count of 'c': positions 3, 11 $\Rightarrow$ 2 occurrences.
- Count of 'd': position 7 $\Rightarrow$ 1 occurrence? Wait, position 7 is: Index 0: a, 1: b, 2: a, 3: c, 4: a, 5: b, 6: a, 7: d, 8: a, 9: b, 10: a, 11: c, 12: a, 13: b, 14: a, 15: e. Wait! 'd' appears at index 7. Does 'd' appear anywhere else? No, 'd' appears only at index 7! Wait, let us re-verify: Indices: 0:a, 1:b, 2:a, 3:c, 4:a, 5:b, 6:a, 7:d, 8:a, 9:b, 10:a, 11:c, 12:a, 13:b, 14:a, 15:e. The characters are: 'a' (8), 'b' (4), 'c' (2), 'd' (1), 'e' (1). The **first** non-repeating character encountered in the string is 'd' at index 7.

Thus, the index of the first character with frequency 1 is 7.

Practice Q5. Subarray Sum Equals Zero via Prefix Hash Map [GATE DA Standard • NAT • 2 Marks] Consider the array $A = [3, 2, -5, 1, 4, -5, 2]$. Using a hash set to store running prefix sums (with initial prefix sum 0), how many distinct contiguous subarrays sum to exactly 0?

Answer: 3

Solution / Explanation:

Let us calculate prefix sums $P[k] = \sum_{i=0}^{k-1} A[i]$ with $P[0] = 0$:

- $P[0] = 0$
- $P[1] = 0 + 3 = 3$
- $P[2] = 3 + 2 = 5$
- $P[3] = 5 + (-5) = 0$
- $P[4] = 0 + 1 = 1$
- $P[5] = 1 + 4 = 5$
- $P[6] = 5 + (-5) = 0$
- $P[7] = 0 + 2 = 2$

A subarray $A[i \dots j]$ sums to 0 if and only if $P[j + 1] = P[i]$. Let us group indices by prefix sum value:

- Value 0 occurs at indices $\{0, 3, 6\}$: $\binom{3}{2} = 3$ pairs:
 1. $(0, 3) \Rightarrow A[0..2] = [3, 2, -5]$, sum = 0.
 2. $(3, 6) \Rightarrow A[3..5] = [1, 4, -5]$, sum = 0.
 3. $(0, 6) \Rightarrow A[0..5] = [3, 2, -5, 1, 4, -5]$, sum = 0.
- Value 5 occurs at indices $\{2, 5\}$: $\binom{2}{2} = 1$ pair:
 4. $(2, 5) \Rightarrow A[2..4] = [-5, 1, 4]$, sum = 0.
- Other values: 3 (1), 1 (1), 2 (1) produce 0 pairs.

Total subarrays summing to zero = $3 + 1 = 4$.

Practice Q6. Greedy Best-First Search Path Selection Tracing**[GATE DA****Standard • MCQ • 2 Marks]** Consider a graph search where the start node is S and the goal is G .

The heuristic estimates to G are: $h(S) = 10, h(A) = 7, h(B) = 3, h(C) = 1, h(D) = 6, h(G) = 0$. The edges and their actual step costs are:

- $S \rightarrow A$ (cost 2), $S \rightarrow B$ (cost 8)
- $A \rightarrow C$ (cost 10), $A \rightarrow D$ (cost 3)
- $B \rightarrow G$ (cost 4)
- $C \rightarrow G$ (cost 5), $D \rightarrow G$ (cost 8)

Under Greedy Best-First Search (using evaluation function $f(n) = h(n)$), which path is chosen to reach the goal G ?

- (A) $S \rightarrow B \rightarrow G$
- (B) $S \rightarrow A \rightarrow C \rightarrow G$
- (C) $S \rightarrow A \rightarrow D \rightarrow G$
- (D) $S \rightarrow C \rightarrow G$

Answer: (A)**Solution / Explanation:**

Trace of Greedy Best-First Search ($f(n) = h(n)$):

1. Expand start node S . Successors: A with $h(A) = 7$, and B with $h(B) = 3$.
2. Priority queue contains $\{B : 3, A : 7\}$.
3. The algorithm greedily expands the node with lowest heuristic value, which is B ($h = 3$).
4. Successors of B : G with $h(G) = 0$.
5. The node with lowest h is now G ($h(G) = 0$). G is selected and goal test succeeds.
6. The path returned is $S \rightarrow B \rightarrow G$.

Note: The path cost is $8 + 4 = 12$. (Meanwhile, path $S \rightarrow A \rightarrow D \rightarrow G$ has cost $2 + 3 + 8 = 13$, and $S \rightarrow A \rightarrow C \rightarrow G$ has cost $2 + 10 + 5 = 17$). GBFS finds $S \rightarrow B \rightarrow G$.

Practice Q7. One-Sentence Admissibility Invariant Verification**[GATE DA Standard •****MSQ • 2 Marks]** Let $h^*(n)$ be the optimal cost from node n to a goal state. Which of the following statements correctly reflect the conditions and implications of **admissibility**?

- (A) A heuristic $h(n)$ is admissible if and only if $0 \leq h(n) \leq h^*(n)$ for all nodes n .
- (B) If $h_1(n)$ and $h_2(n)$ are both admissible, then $h_3(n) = \max(h_1(n), h_2(n))$ is also admissible and dominates both.
- (C) If $h(n)$ overestimates the true cost for even a single non-goal state, A* tree search is no longer guaranteed to return the optimal solution.
- (D) The trivial heuristic $h(n) = 0$ for all nodes is admissible.

Answer: (A, B, C, D)

Solution / Explanation:

- (A) is the foundational definition of admissibility (optimism without overestimation).
- (B) is True: Since $h_1(n) \leq h^*(n)$ and $h_2(n) \leq h^*(n)$, $\max(h_1(n), h_2(n)) \leq h^*(n)$, and higher heuristic values prune more nodes.
- (C) is True: If $h(n)$ overestimates an optimal path, A* may prematurely explore and return an alternative sub-optimal path whose estimated cost appears smaller.
- (D) is True: Since true costs $h^*(n) \geq 0$, setting $h(n) = 0 \leq h^*(n)$ is valid (reducing A* to Dijkstra's / Uniform-Cost Search).

Practice Q8. Load Factor and Average Successful Search Time

[GATE DA

Standard • MCQ • 2 Marks] Under the assumption of uniform hashing in an open-address hash table with load factor $\alpha = \frac{n}{m} = 0.5$, what is the theoretical upper bound on the average number of probes required in an **unsuccessful** search?

- (A) 1.5
- (B) 2.0
- (C) 0.5
- (D) 4.0

Answer: (B)

Solution / Explanation:

In open addressing under uniform hashing, the expected number of probes in an unsuccessful search is:

$$E[\text{probes}] = \frac{1}{1 - \alpha}$$

Substituting $\alpha = 0.5$:

$$E[\text{probes}] = \frac{1}{1 - 0.5} = \frac{1}{0.5} = 2.$$

Practice Q9. Dominance Among Admissible Heuristics **[GATE DA Standard • MCQ • 1 Mark]**

Given two admissible heuristics h_A and h_B such that for all states n , $h_A(n) \geq h_B(n)$. Which of the following conclusions is correct regarding A* search using these heuristics?

- (A) h_B dominates h_A and expands fewer nodes.
- (B) h_A dominates h_B and will expand at most as many nodes as h_B (never more).
- (C) h_A is inadmissible because its values are strictly greater.
- (D) Both heuristics will necessarily expand the exact same number of nodes.

Answer: (B)

Solution / Explanation:

By definition, if $h_A(n) \geq h_B(n)$ for all nodes n , h_A is said to **dominate** h_B . A more informed (closer to true cost h^*) admissible heuristic prunes a larger portion of the search space, meaning A* with h_A expands no more nodes than A* with h_B .

Practice Q10. Hash Collisions in Python Dictionaries [GATE DA Standard • MSQ • 2 Marks]

Which of the following statements is/are **TRUE** regarding hash maps and Python dictionaries?

- (A) In Python, if two distinct objects x and y have `hash(x) == hash(y)`, they must satisfy $x == y$.
- (B) If two objects satisfy $x == y$, Python requires that `hash(x) == hash(y)`.
- (C) Python dictionaries internally resolve collisions using open addressing with pseudo-random perturbation probing.
- (D) Insertion of n elements into a hash table takes $\mathcal{O}(1)$ time in the absolute worst case.

Answer: (B, C)

Solution / Explanation:

- (A) is False: Due to the pigeonhole principle, distinct objects can produce the same hash integer (a hash collision).
- (B) is True: The hash invariant in Python states that objects that compare equal must have equal hash values.
- (C) is True: CPython's dictionary implementation uses open addressing with a perturbation recurrence formula ($i \leftarrow (5i + 1 + \text{perturb}) \bmod 2^k$).
- (D) is False: In the worst case where all elements collide into the same slot, total insertion time can degrade to $\mathcal{O}(n^2)$ (or $\mathcal{O}(n)$ per insertion).

End of Question Bank — GATE Data Science & Artificial Intelligence (DA)